

DEEP LEARNING

Labwork 3

Students Ho Thanh Thuy Tien - 2540100

1 Implementation

Model and Sigmoid Function

Logistic regression is a binary classification algorithm that models the probability that a sample belongs to a given class. For this dataset with two input features x_1 (salary) and x_2 (experience), we will have the linear combination as below:

$$z = w_0 + w_1x_1 + w_2x_2$$

This raw score is passed through the **sigmoid** (logistic) function to produce a probability in $(0, 1)$:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss Function

I used the **Binary Cross-Entropy** (log loss) as the objective to minimize:

$$J = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where N is the number of training samples and $y_i \in \{0, 1\}$ is the true label.

1.1 Gradient Descent Update Rules

The gradients of the loss with respect to each weight are derived using the chain rule. A key property of the sigmoid function is that its derivative simplifies to $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, which makes the gradient expressions clean:

- Partial Derivative for w_0 :

$$\frac{\partial J}{\partial w_0} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)$$

- Partial Derivative for w_1 :

$$\frac{\partial J}{\partial w_1} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \cdot x_{1,i}$$

- Partial Derivative for w_2 :

$$\frac{\partial J}{\partial w_2} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \cdot x_{2,i}$$

The weights are updated iteratively using a learning rate r (w_1, w_2 also have the same function):

$$w_0 = w_0 - r * \frac{dJ}{dw_0}$$

And the gradient stop iterating when the change in the loss between two consecutive steps fell below a specific threshold (which is $1e-6$).

2 Learning Rate Analysis

I tested the model with three different learning rates (r).

- **Low Learning Rate** ($r = 0.001$): It took **76,146 iterations** to converge. The final loss of 0.34601 is the highest among the three settings. Also, the final decision boundary is noticeably weaker than others.
- **Moderate Learning Rate** ($r = 0.03$): We achieved convergence in only 24,916 iterations, making it roughly 3× faster than the low rate. At the same time, the loss dropped significantly to 0.21656.
- **High Learning Rate** ($r = 0.5$): It just took **1,415 iterations** to converge, the fastest by a large margin, and reached the lowest loss of 0.20961. The weights are similar to those of $r = 0.03$, this mean this convergence to the same region.

In summary, a higher learning rate dramatically reduces the number of required iterations. However, if the rate is set too high (beyond the stability threshold for the dataset), the loss will start to oscillate or diverge instead of decreasing. For this loan decision dataset, $r = 0.5$ was still stable and reached the best result in the fewest iterations.

3 Visualization

Next, I visualized the loan decision dataset, with salary on the x-axis and experience on the y-axis. Red dots represent applicants who received a loan (*Loan*) and blue dots represent those who were refused (*Refuse*). The green line is the **decision boundary** produced by logistic regression with $r = 0.5$, defined by:

$$w_0 + w_1 \cdot \text{salary} + w_2 \cdot \text{experience} = 0$$

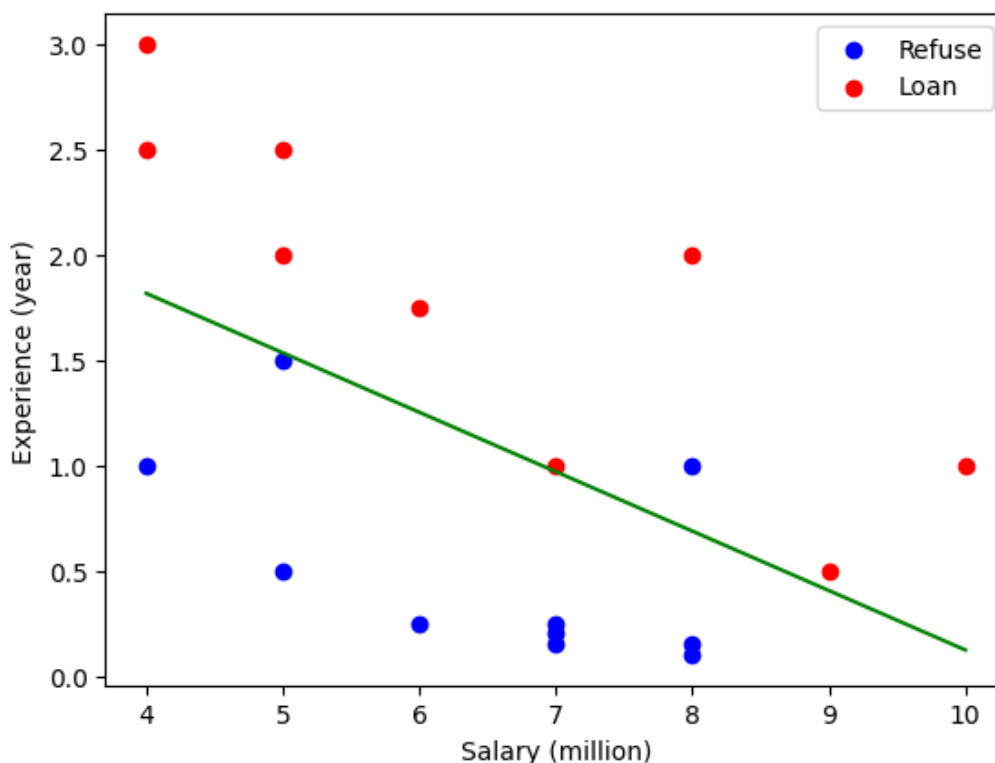


Figure 1: Logistic regression decision boundary on the loan decision dataset ($r = 0.5$, threshold = 10^{-6})

The boundary separates the two classes pretty well even though there is some overlap in the middle of the salary range.